

Overview

At dubizzle, we build practical, intelligent systems that solve real world user problems. One of our key verticals is the used cars marketplace (**dubizzle cars**), where users constantly inquire about buying, selling, and exploring vehicle listings.

Your task is to build a **working prototype of an AI Assistant** that helps an end user explore a provided inventory of car listings and have a smart, contextual conversation about them.

Furthermore, the agent should recognize returning users and remember their previous preferences and inquiries.

The Challenge

You need to build a **FastAPI backend service** paired with a client interface (*StreamLit* or *Jupyter Notebook*) of your choice. We are evaluating your ability to integrate **language models with data retrieval techniques, API design**

Core Capabilities Required

Capability	Description & Requirements
Inventory Search & Retrieval	We will provide a mock dataset of ~100 car listings (containing Make, Model, Trim, Year, Price, Description, etc.). The agent must be able to understand a user's request, query this dataset via the backend, and present matching cars.
Book Car Viewing Slots & Qualify Leads	dubizzle managed cars can be viewed and test driven provided it is prebooked, the available time slots for all cars is Monday to Saturday (8am to 8pm). Additionally, the agent should gather information on the user's price range, and other needs and save them to a local CSV simulating leads being recorded and qualified.
Short-Term Memory	The agent must remember details within the current session. If the user asks " <i>What's the mileage on that first Honda?</i> " and follows up with " <i>Is there a warranty on it?</i> ", the agent must maintain context without making the user

Capability	Description & Requirements
	restate the vehicle.
Long-Term Memory (Cookie Points for Doing this)	The agent should be able to identify a returning user (e.g., via a user ID or name prompt) and recall information from previous sessions. For example, remembering the user's name, that they were previously looking for a "white SUV under \$20k", or recalling a car they liked yesterday.
Intent Recognition & Guardrails	The agent should handle general chit-chat, inventory queries, and simulated tool use (e.g., booking a test drive). It must gracefully and politely decline non-automotive requests and avoid referring to competitors (e.g., writing code or answering history questions, talking about other used car platforms).

Technical Scope & Stack

We want to see how you leverage the modern tooling ecosystem (e.g., uv for package management, FastAPI, LiteLLM and other frameworks) and stitch them together elegantly.

LLM Access

No Out-of-Pocket Costs: To make this assignment frictionless, create and use a an API key from

Get an API key | Google AI Studio

You can use the free tier.

DATASET

Attached as a separate file

What You Need to Build

- The FastAPI Backend (main.py):** A robust API service that exposes endpoints for chat processing, state persistence, and inventory retrieval.
- Data Retrieval Module:** A mechanism managed by your backend (e.g., vector database for

RAG, text-to-SQL, or function calling/tools) to search and filter the provided car inventory based on natural language queries.

3. **The Persistent Storage Module:** A basic mechanism (SQLite, JSON, or a specialized memory DB) to store and retrieve long-term user profiles and preferences across API calls.
4. **The Client Interface (Your Choice):** A frontend layer that interacts with your FastAPI backend. You can choose to build **either**:
 - **Option A:** A local Jupyter Notebook (client.ipynb) utilizing `httpx` or `requests` to step through the conversation.
 - **Option B:** A lightweight **Streamlit application** (`app.py`) providing a clean, reactive UI chat interface.

Engineering Pro-Tip

We highly recommend setting up your workspace using "uv" as your project and package manager. It makes running local FastAPI servers and managing environment dependencies (like Jupyter kernels or Streamlit) incredibly fast and seamless.

It also allows us to run your code to test locally!

You can choose how to tackle the dataset search. Because the dataset contains both structured data (Year, Price, Make) and unstructured data (Description), you might consider using standard SQL/pandas Tool Calling, or a hybrid Retrieval-Augmented Generation (RAG) approach. Choose the method you think yields the most accurate conversational search results!

What We Are Evaluating

We care about your engineering intuition, system control, and resourcefulness.

- **Architecture & API Design:** How cleanly is the boundary separation maintained between your FastAPI core logic and your chosen client layer?
- **Data Retrieval & Grounding:** How accurately does the agent search the provided dataset and return relevant cars without hallucinating inventory?
- **State & Memory Management:** How elegantly is short-term multi-turn conversation handled, and how effectively does the agent recall long-term user preferences across simulated sessions?
- **Prompt Engineering & Control:** Does the agent stick strictly to the factual details provided in the dataset, and properly decline out-of-scope requests?

Submission Guidelines

Deliverable: A public GitHub repository (or zip file) containing your code, a requirements.txt or pyproject.toml (ideally utilizing the uv layout), and a short README.md.

- The project should run without errors!
- **The README should include:**
 - Quick setup and execution instructions for both the backend server and your chosen interface.
 - A brief paragraph on **why** you chose your specific client setup (Notebook vs. Streamlit), agent framework, search retrieval method, and memory storage.
 - A short explanation of the implementation in 2 short paragraphs outlining design decisions and what features could be implemented that are outside the scope of this project.
 - Screenshots or terminal logs showing:
 1. A successful multi-turn conversation exploring the inventory.
 2. A demonstration of the agent recalling a user's previous preferences or history in a completely new "session".